Mini-Project 2: Flow Matching Parameterisation

Mayukh Das - U7965027 - COMP4680/ENGN8650, ANU

1. Implementation and experimental setup

All experiments use a 5-layer MLP with 256 hidden units per layer and ReLU activations, taking z_t concatenated with a 128-dimensional sinusoidal time embedding as input. The forward process is defined as

$$z_t = (1-t)\,x + t\,\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \qquad (1)$$

and the velocity parameterisation is v = ε − x. Experiments were conducted on swiss_roll, gaussians, and circles datasets at $D \in \{2,8,32\}$. Training uses Adam with learning rate 1e-3 and batch size 1024. Parts 1–3 were trained for 25,000 steps, while Part 4 used 50,000 steps due to the increased optimisation difficulty of MeanFlow. To avoid numerical instability from division by small timesteps, t is sampled uniformly in $[\varepsilon, 1-\varepsilon]$. I used $\varepsilon = 5 \times 10^{-2}$ for Parts 1–3 and $\varepsilon = 1 \times 10^{-3}$ for Part 4.2. Sampling uses Euler ODE integration with 50 steps as specified in the assignment. For x-prediction models, predicted clean data x̂ is converted to velocity via v = (z − x̂)/t.

For MeanFlow (Part 4.2), the model additionally takes the horizon input h = t − r using a second sinusoidal embedding as required by the specification. The model predicts x̂, with the average velocity recovered as u = (z − x̂)/t. The target is computed using per-sample Jacobian-vector products via torch.func.jvp with tangent vector (v, 0, 1) for inputs (z, r, t), which matches Algorithm 1 from the MeanFlow paper. Training uses a flow matching ratio of 0.5, meaning 50% of samples use standard flow matching with h = 0, while the remaining 50% use MeanFlow training with h > 0. Adaptive loss weighting with p = 1.0 is applied to both the flow matching and MeanFlow losses.

2. Part 1: Warm-up

I used the recommended hyperparameters above. v-prediction with v-loss at D=2 recovers all three datasets cleanly. The spiral, the rings and the 8-mode mixture all show up as expected and passed the sanity check.

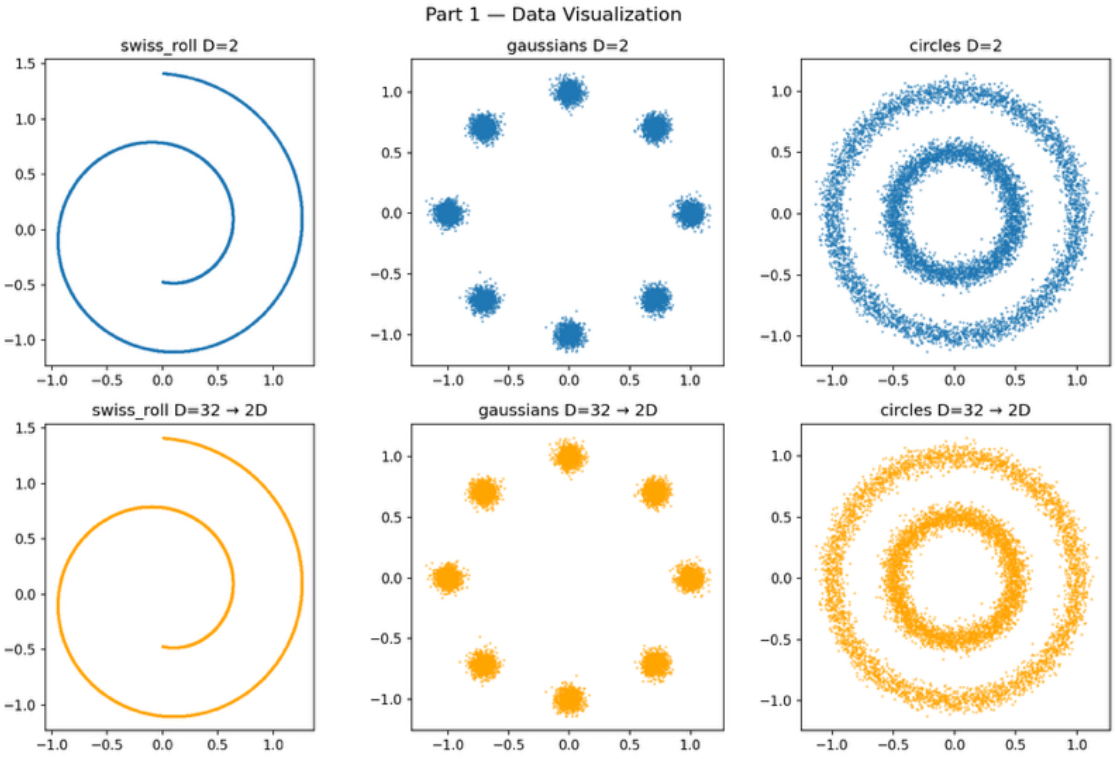


Figure 1: 6-panel visualisations of original 2D and back-projected 32D for swiss_roll, gaussians, circles

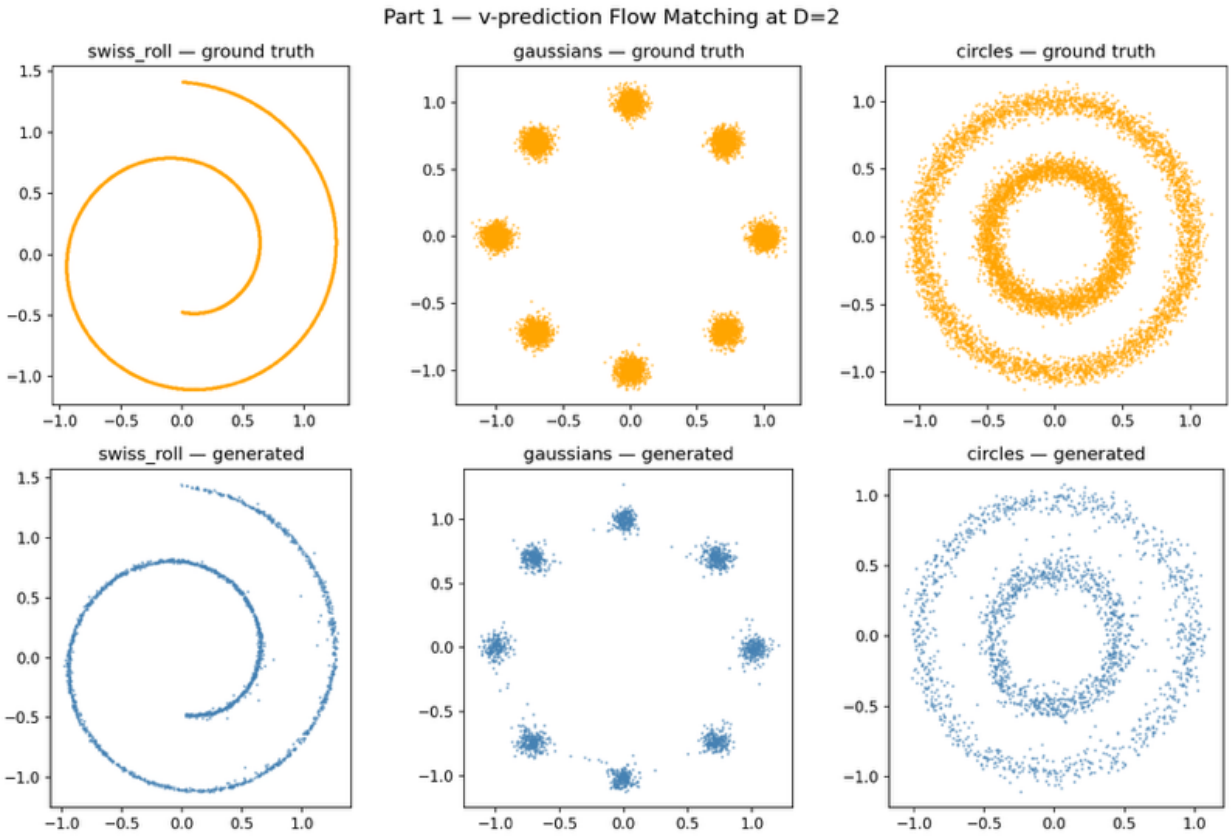


Figure 2: 3-panel D=2 v-pred v-loss generated samples vs ground truth

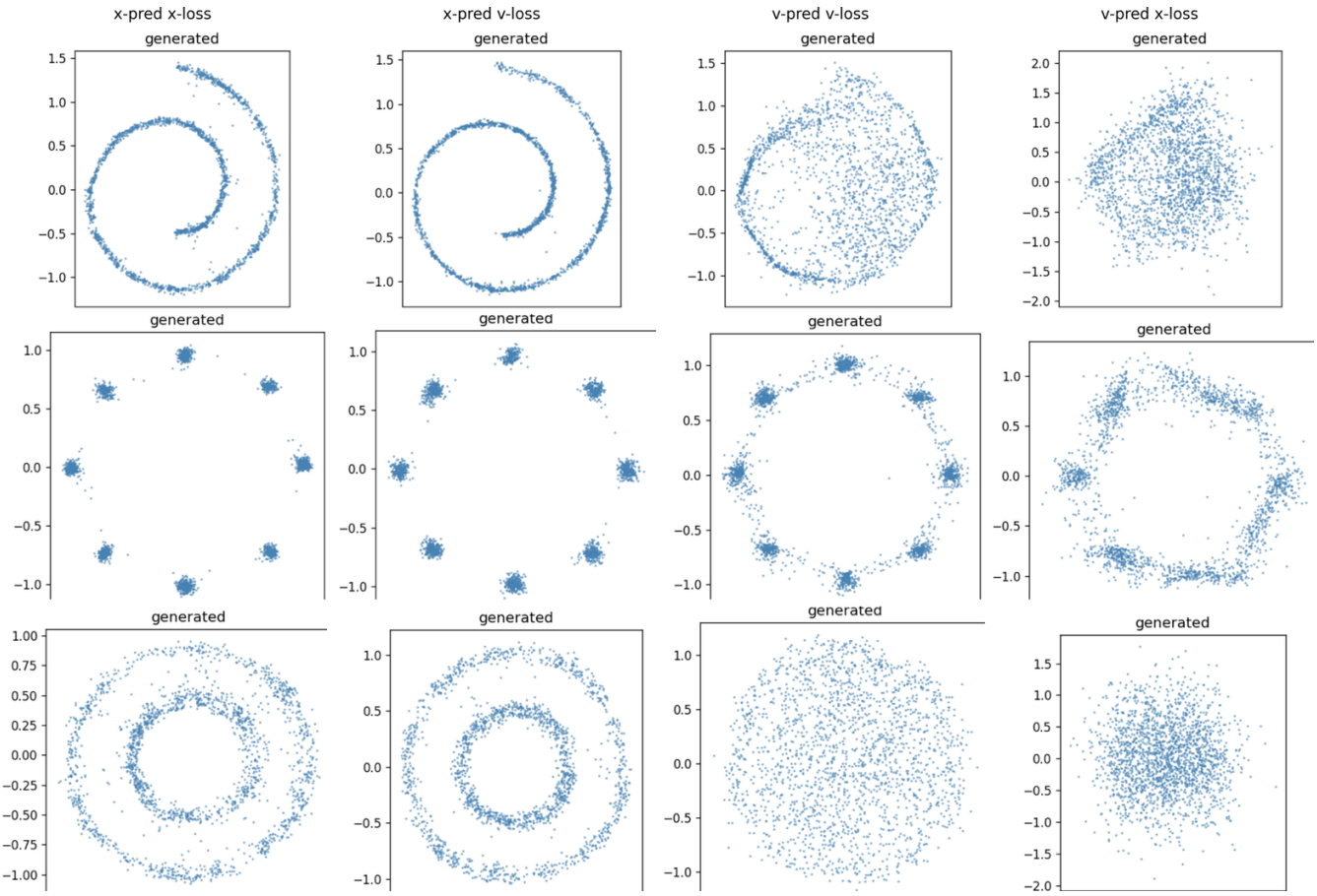


Figure 3: D=32 parameterisation comparison. Rows show datasets and columns show prediction/loss combinations. x-prediction remains stable while v-prediction fails at high dimension.

3. Part 2: Parameterisation

Conversion. From $z_t = (1-t)x + t\varepsilon$ we have $\varepsilon = (z_t - (1-t)x)/t$, so

$$v = \varepsilon - x = \frac{z_t - x}{t}, \tag{2}$$

and solving for x gives $x = z_t - tv$. The two parameterisations are linearly related given (z_t, t).

I trained all four predictions × loss combinations on the three datasets at D ∈ {2, 8, 32}, 36 runs total.

Combo	D=2	D=8	D=32
x-pred + x-loss	good	good	good
x-pred + v-loss	good	good	good
v-pred + x-loss	good	degrades significantly	fails catastrophically
v-pred + v-loss	good	degrades slightly	fails

Q1. x-prediction works robustly at all dimensions. Both x-pred configurations continue to generate clean samples at D=32, preserving the geometry of circles, swiss_roll, and gaussian mixtures. In contrast, v-prediction begins degrading at D=8, producing thicker swiss_roll structure, partial mode coverage on gaussians, and noisier circles. At D=32, both v-prediction configurations fail completely and no longer recover recognisable dataset structure.

Q2. Loss type has a secondary but noticeable effect. In both x-pred and v-pred, using the loss that matches the prediction target performs slightly better. In particular, x-pred + x-loss performs best overall, while v-pred + x-loss performs worst.

However, the dominant effect is still the prediction target itself rather than the loss formulation. Both x-prediction configurations maintain stability at D = 32, while both v-prediction configurations fail. This suggests that loss alignment of pred and loss improves optimisation conditioning but does not fundamentally change the difficulty of the underlying regression target.

Q3. The asymmetry comes from the rank of each target relative to the ambient dimension. The clean data x still lies on a simple 2D structure even after projection into R^D, so x-prediction only needs to learn a relatively simple target regardless of D. A 256-hidden-unit MLP is enough to model this structure even at D=32.

In contrast, v = ε − x adds a full-dimensional Gaussian noise vector to the x that lies on the lower dimensional manifold. As D increases prediction has to approximate a target whose effective dimension scales the entire ambient space.At fixed model capacity, the network can no longer approximate this target accurately enough for stable Euler sampling, causing v-prediction to fail at high dimensions.

This matches the main observation from the JiT paper: x-prediction benefits from the low-dimensional structure of the data, while v-prediction must model the full noisy ambient space.

4. Rescuing v-prediction (Part 3)

I engaged with the RAE paper's broader argument about the relationship between parameterisation, ambient dimension, target complexity, and model capacity, rather than attempting to directly verify RAE's specific d ≥ n threshold rule. In RAE, n = 768 refers to the token channel dimension, while in our setting the closest analogue is the data dimension D = 32. Since the baseline width d = 256 already exceeds this by a factor of 8, I used RAE primarily as conceptual motivation rather than as a threshold to reproduce exactly.

Following Part 2 Q3, v-prediction fails at D = 32 because its target scales with the full ambient dimension, while x-prediction only needs to model the underlying low-dimensional structure of the data. A natural rescue strategy is therefore to increase model capacity so the network can better approximate the harder v-prediction target. Inspired by Sections 3 and 4 of RAE, I tested two interventions: increasing model width and shifting the noise schedule.

Using swiss_roll at D = 32 with v-prediction + v-loss, I ran a 7-cell sweep varying hidden width h ∈ {256, 512, 1024} under both the default and shifted schedules, together with an x-prediction reference model at h = 256. The full width × schedule sweep appears in Appendix B.

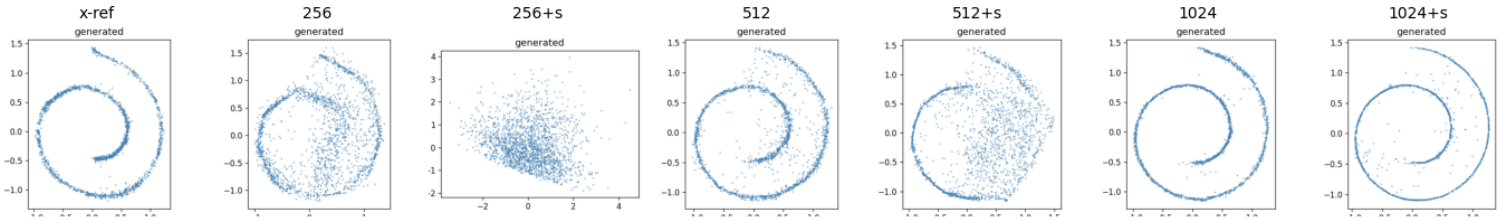


Figure 4: Width × schedule sweep for v-prediction on \texttt{swiss_roll} at D = 32. Increasing width rescues v-prediction, while the shifted schedule alone does not improve stability.

Increasing width to h = 1024 under the default schedule substantially improves v-prediction quality, producing a clean spiral visually comparable to the x-prediction reference. In contrast, the schedule shift alone does not help and actually worsens performance at low width. The combination of h = 1024 and the shifted schedule is slightly noisier than the default h = 1024 run, suggesting that the two interventions do not combine constructively.

Q1 — Is v-prediction's failure fundamental or can it be overcome? (2 marks)

Not fundamental. v-prediction is rescuable by widening the network from h = 256 to h = 1024. This supports rather than contradicts our Part 2 observation: v-prediction failed because its target has higher effective dimension than x-prediction's and providing additional model capacity is sufficient to fit the harder target. The issue is therefore a capacity limitation rather than a structural impossibility.

Q2 — What did you try? Compute cost vs default x-prediction? (3 marks)

I tested two interventions:

1. increasing hidden width h ∈ {256, 512, 1024} under the default schedule.
2. applying the shifted schedule from RAE Section 4.2 at each width.

Only widening to h = 1024 successfully rescued v-prediction to x-prediction-comparable quality. The compute cost is substantially larger: the h = 256 model contains roughly 330K parameters, while the h = 1024 model contains roughly 5.2M parameters, an increase of approximately 16x. Since FLOPs scale roughly with parameter count for the MLP layers, x-prediction achieves comparable quality far more efficiently in this toy setting.

Q3 — Compare how x-prediction and v-prediction respond to your changes. (3 marks)

x-prediction is largely insensitive to width in our experiments. The underlying 2D structure of the dataset is already easy to fit at h = 256, and increasing width to h = 1024 produces little visible improvement.

v-prediction behaves very differently. At h = 256, sampling fails completely, while at h = 1024 the model generates clean spirals. This asymmetry matches the explanation from Part 2 Q3: x-prediction only needs to

model the simple low-dimensional structure underlying the data, whereas v-prediction must model a target whose complexity scales with the full ambient dimension. Additional capacity therefore matters much more for v-prediction than for x-prediction.

Q4 — Why does v-prediction work in real systems like SD3 and FLUX? (7 marks)

The toy regime used in this assignment is very different from the setting used by modern diffusion systems. In our experiments, x-prediction benefits from the fact that the data lies on a simple low-dimensional structure embedded inside R^32. That structural advantage largely disappears in production diffusion models.

First, modern systems typically operate in compressed latent spaces produced by VAEs or representation encoders, where the data is no longer concentrated on a simple low-dimensional structure. In that regime, both x-prediction and v-prediction targets behave more like full-dimensional regression problems, so the low-dimensional advantage of x-prediction no longer applies.

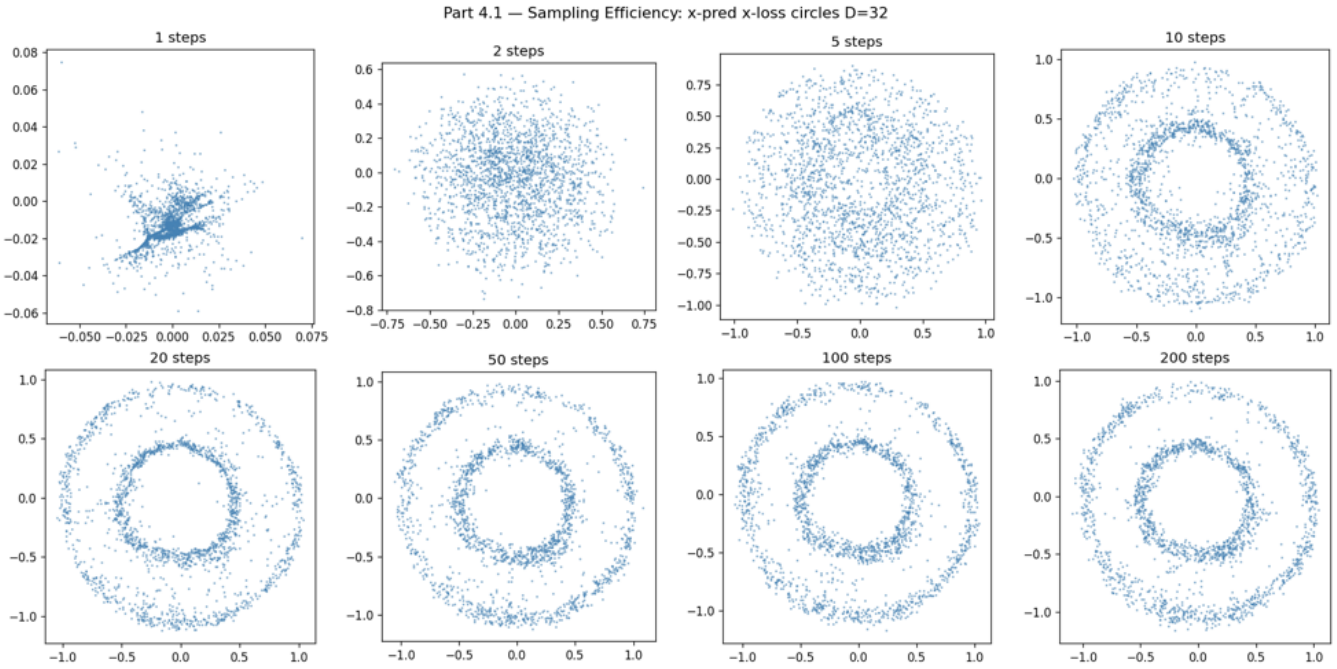
Second, v-prediction tends to maintain more stable regression difficulty across timesteps. At large t, x-prediction requires reconstructing clean data from almost pure noise, which becomes increasingly difficult. In contrast, the difficulty of the v-prediction target remains more consistent across timesteps. In large-scale systems trained for extremely high sample quality, this more stable optimisation behaviour becomes important and can outweigh the low-dimensional advantages observed in our toy experiments. The JiT paper itself acknowledges this calibration argument and frames its preference for x-prediction as conditional on the data having exploitable low-rank structure.

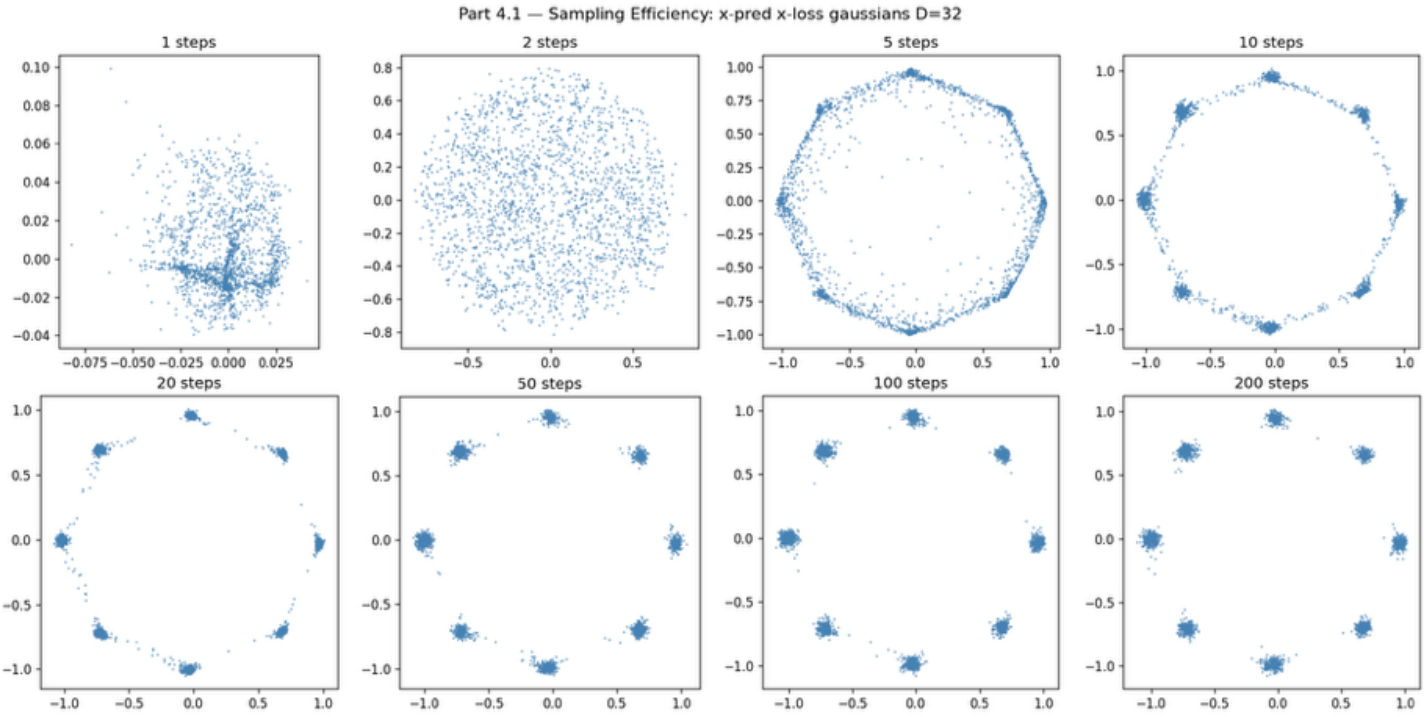
Our results therefore do not contradict the use of v-prediction in systems such as SD3 or FLUX. Instead, they identify a specific regime where x-prediction has a structural advantage: low-dimensional data embedded in a high-dimensional ambient space, relatively small models, and limited training budgets. In this regime, x-prediction can exploit the underlying structure of the data, while v-prediction must still model full-dimensional noise. As model capacity and training scale increase, and as modern systems move to dense representations rather than sparse low-rank manifolds, this advantage weakens. Under those conditions, the more uniform timestep difficulty and optimisation behaviour of v-prediction become increasingly important, which helps explain its success in large-scale diffusion systems.

5. Sampling efficiency (Part 4.1)

Using x-prediction + x-loss (the best-performing configuration from Part 2) at D = 32, I evaluated sample quality across Euler step counts

{1,2,5,10,20,50,100,200} on all three datasets at D = 32 to match Part 4.2.





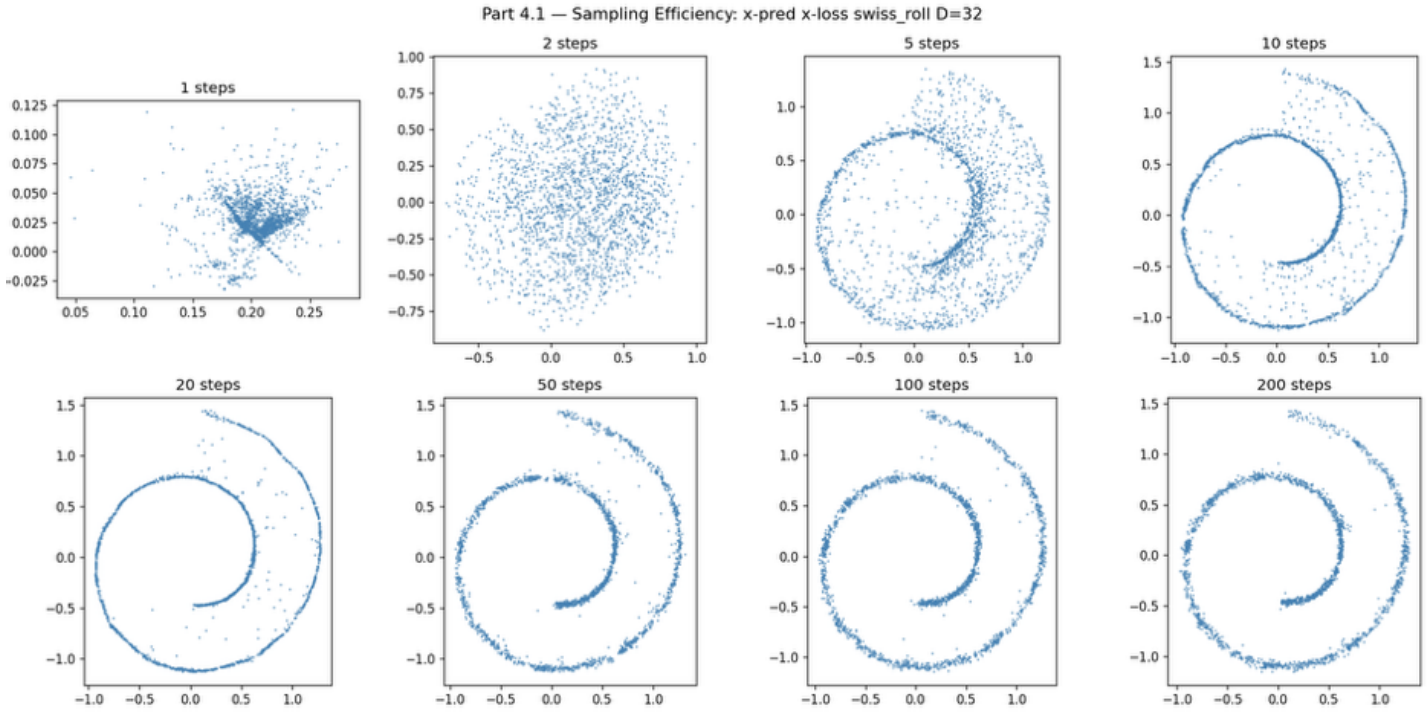


Figure 5: per-dataset summary grids showing degradation across step counts

Observations

Recognisable distributional structure emerges by approximately 10 sampling steps across all three datasets. Sample quality improves rapidly between 1 and 20 steps, largely saturating between 20 and 50 steps; increasing beyond 50 produces marginal visual improvement. At 1 step, x-prediction collapses to a single forward pass. The model captures overall structure but loses fine detail: the swiss_roll retains spiral structure but loses clean trajectory separation, gaussian modes collapse toward a noisy central mass with only weak modal separation, and the circles lose clear ring separation. At 2–5 steps, the samples improve substantially. The swiss spiral becomes recognisable, gaussian modes begin separating, and the circles recover distinct rings. These results provide the baseline against which MeanFlow's few-step generation will be compared in Part 4.2.

6. MeanFlow (Part 4.2)

Implementation

I implemented MeanFlow using x-prediction, chosen because Part 2 showed that x-prediction scales reliably to D = 32 while v-prediction does not. The model predicts clean data $\hat{x}$, from which the average velocity is derived as $u = (z - \hat{x})/t$. The Meanflow target

$$u_{\text{target}} = v - (t - r)\,\frac{\partial u}{\partial t} \qquad (3)$$

is computed using per-sample Jacobian-vector products via torch.func.jvp, wrapped in vmap and functional_call following Algorithm 1 from the paper. I used a flow_ratio of 0.5, meaning 50% of each batch trains as standard flow matching with h = 0, while the remaining 50% uses MeanFlow training with h > 0.

After ablation (discussed in Q5), I adopted adaptive loss weighting (p = 1.0) but not the logit-normal timestep sampler; both t and r are sampled uniformly. MeanFlow was trained for 50,000 steps, longer than Parts 1–3 because the bootstrap target is noisier and more difficult to optimise. Per-dataset random seeds were selected using 10-seed sweeps due to the seed sensitivity discussed later (0 for swiss_roll, 100 for gaussians, 456 for circles).

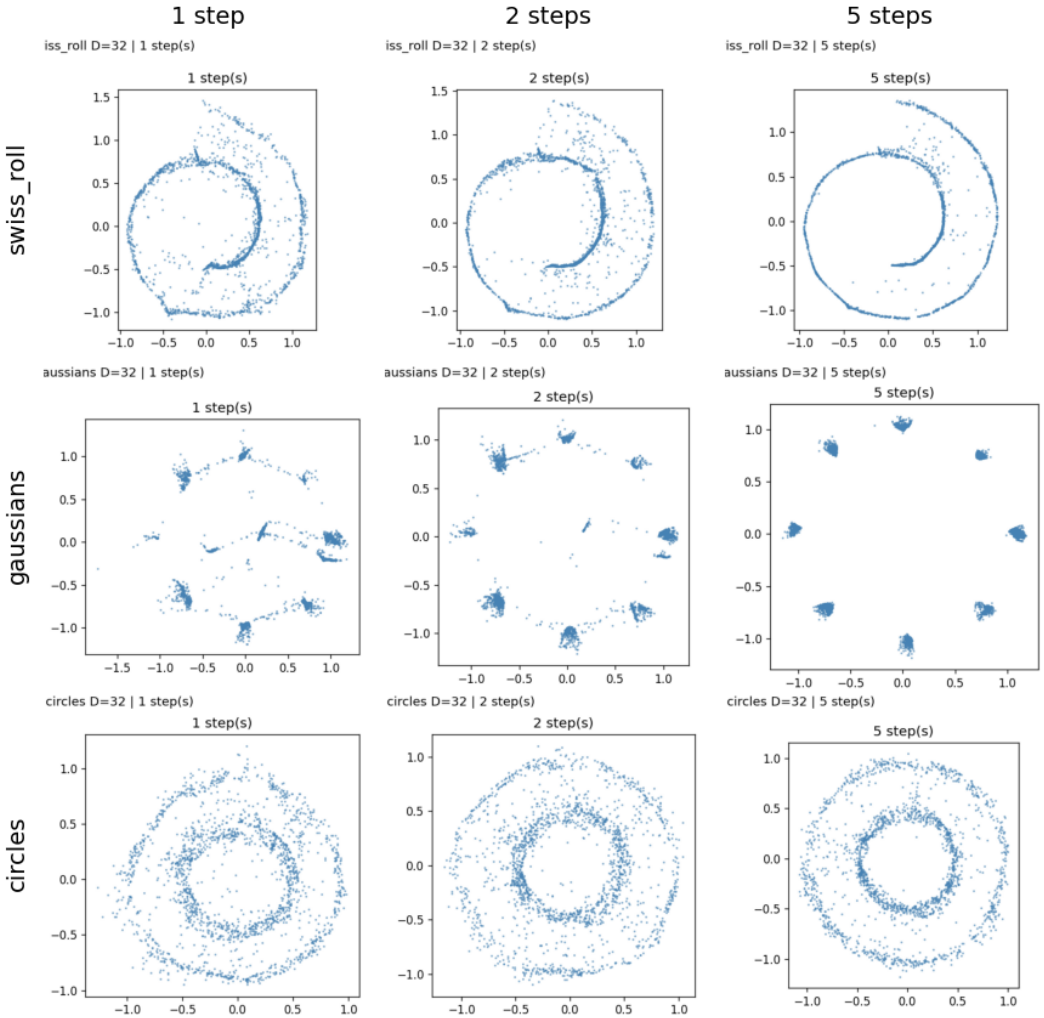


Figure 6: 9-panel grid — 3 datasets × {1, 2, 5} MeanFlow steps

Q1 — Why x-prediction for MeanFlow? (2 marks)

I used x-prediction because Part 2 showed that it remains stable at D = 32, whereas v-prediction does not. Using v-prediction directly inside MeanFlow would inherit the same high-dimensional target problem identified earlier: the network would still need to learn a full-dimensional noisy regression target during the h = 0 flow-matching portion of training. By predicting x instead and converting to velocity through u = (z − x̂)/t, the model can exploit the lower-dimensional structure of the data while still producing the velocity quantities required by MeanFlow.

Q2 — Core idea of MeanFlow. What does the model learn vs standard FM. Why does this enable one-step generation? (4 marks)

Standard flow matching learns the instantaneous velocity field v(z_t,t), which describes only the local direction of motion at a single timestep. Sampling therefore requires many small Euler updates to numerically integrate the trajectory from noise to data.

MeanFlow instead learns the average velocity over a finite interval:

$$u(z_t, r, t) = \frac{1}{t-r} \int_r^t v(z_\tau, \tau)\, d\tau. \tag{4}$$

Rather than predicting infinitesimal motion, the model predicts the net velocity across part of the trajectory. Integrating the flow equation gives

$$z_t - z_r = (t-r)\, u(z_t, r, t), \tag{5}$$

so, when r = 0 and t = 1, a single prediction of u directly estimates the full displacement from noise to data.

MeanFlow trains this average-velocity field using a target involving u and ∂u/∂t. Jacobian-vector products (JVP) efficiently compute the required derivative term ∂u/∂t per sample, allowing the model to learn the average velocity field without explicitly simulating trajectories during training.This allows one-step generation because the model learns the integrated transport itself rather than approximating it through many small sampling steps.

Q3 — Why is the h = 0 portion needed? (3 marks)

At h = 0, the average velocity reduces to the standard instantaneous velocity, $u(z,t,0) = v(z,t)$. This means the model receives ordinary flow-matching supervision at h=0. This supervision is important for two reasons. First, the MeanFlow target for h>0 depends on derivatives of the learned average-velocity field, and those derivatives are only meaningful if the underlying velocity estimates are already reasonably accurate. Second, the h>0 targets depend partly on the model's own current predictions. Without the h=0 supervision, the model would effectively be training against self-generated targets with no direct grounding to the true velocity field, making optimisation unstable and prone to collapse. The h=0 portion therefore acts as the anchor that stabilises MeanFlow training.

Q4 — Training cost vs standard FM. Why is MeanFlow harder to train? JVP overhead per step? (3 marks)

A MeanFlow training step costs roughly 2× more than standard flow matching because of the JVP computation. Estimating both u and ∂u/∂t requires propagating tangent vectors alongside the normal forward pass, making each step approximately equivalent to two forward passes plus the usual backward pass.

MeanFlow is also structurally harder to optimise. The MeanFlow target depends partly on the model's own current predictions through ∂u/∂t meaning the regression target changes throughout training. The model is therefore effectively chasing a moving target rather than fitting a fixed supervision signal. Early in optimisation, when the model predictions are still inaccurate, this target becomes noisy and unstable. Compared to standard flow matching, MeanFlow therefore converges more slowly and is noticeably more sensitive to hyperparameters. I compensated by increasing training length to 50,000 steps and by adopting adaptive loss weighting to prevent occasional large target errors from dominating the gradients.

Q5 — Compare MeanFlow samples against ground truth. Artifacts on gaussians? Describe and explain. (7 marks)

On swiss_roll and circles, MeanFlow recovers the underlying manifold structure cleanly across 1, 2, and 5 sampling steps. The spiral and ring structures are already recognisable after a single forward pass and become sharper with additional steps.

The gaussians dataset behaves differently. The eight modes appear at approximately the correct angular positions, but each mode becomes overly concentrated compared to the ground-truth Gaussian blobs. Faint arcs also appear between neighbouring modes, particularly at step 1. Increasing the number of sampling steps sharpens the mode-centre concentration further rather than recovering the original within-mode spread. This is demonstrated in the gaussians row of Figure 6.

I interpreted this as a structural property of MeanFlow's averaged-velocity target on multi-modal data. MeanFlow trains on velocity averaged over the interval [r, t]. At any point between two modes, the per-mode velocity contributions point in different directions; averaging them yields a target that drives samples toward the geometric centre of nearby modes rather than letting them disperse within a mode. Standard flow matching does not have this issue because each training sample provides a single per-sample velocity target v = ε − x tied to one specific data point — no averaging across modes.

The Part 4.1 baseline at matched step counts (1, 2, 5) shows broader gaussian blobs under ordinary flow matching, supporting the interpretation that the concentration effect is specific to MeanFlow rather than simply a consequence of few-step Euler sampling.

I also observed substantial sensitivity to random seed across all three datasets, particularly in the few-step regime. Different seeds changed local density patterns, mode coverage, trajectory continuity, and the amount of stray off-manifold noise. This sensitivity is expected because the MeanFlow target depends partly on the model's own current predictions and derivatives, so small differences early in training alter the learned averaged-velocity field and compound during few-step sampling. The effect was strongest on the gaussians dataset, where small changes in the velocity field altered mode coverage and attractor behaviour, but similar variation was still visible on swiss_roll and circles. I therefore selected per-dataset seeds using 10-seed sweeps to obtain representative and stable qualitative behaviour. Final per-dataset MeanFlow generations appear in Appendix D. Importantly, the gaussian mode-concentration characteristic persisted across all seeds, suggesting that it reflects a structural property of MeanFlow rather than an isolated optimisation failure.

I additionally performed a 2 × 2 ablation on the two stabilisation techniques recommended by the paper: the logit-normal timestep sampler (Table 1d) and adaptive loss weighting (Table 1e). In our experiments, the logit-normal sampler consistently amplified gaussian mode collapse, often shrinking each Gaussian mode into extremely concentrated point-like clusters, whether applied alone or combined with adaptive weighting. In contrast, adaptive weighting alone improved swiss_roll

and circles without significantly harming gaussian mode coverage, which is why our final implementation adopts adaptive weighting but not logit-normal timestep sampling.

One possible explanation is that the logit-normal sampler concentrates training around intermediate timesteps, reducing supervision at small t where mode separation is most important. In our experiments this caused the Gaussian modes to collapse into highly concentrated point-like clusters rather than maintaining their original spread. This is as an interpretation supported by the plots rather than as a proven property of MeanFlow. The paper evaluates MeanFlow on large-scale image datasets where the data is densely supported with no atomic modes, and the optimisation regime differs substantially from our low-dimensional toy setting. In our experiments, preserving accurate small-t behaviour appears important for maintaining mode separation on the gaussians dataset, while the stability concerns motivating the logit-normal sampler in the paper did not appear to surface here. As a result, the shifted timestep distribution may hurt mode separation in our setting without providing a compensating optimisation benefit.

7. Conclusion

x-prediction succeeds in our toy regime because the data manifold is low-dimensional inside the ambient space; v-prediction can be rescued at ~16x compute and is preferred in production systems where this manifold advantage no longer applies. MeanFlow's JVP-based bootstrapping handles manifold data cleanly in one step, but its averaged-velocity target struggles on multi-modal distributions where per-mode contributions interfere destructively. The paper's stabilisation recipe was not portable: adaptive weighting helped everywhere, but the logit-normal sampler amplified the multi-modal weakness MeanFlow inherits structurally — a reminder that recipes from large-scale work are tuned for failure modes that may not exist in toy regimes.

8. References

Geng, Z., Deng, M., Bai, X., Kolter, J.Z. and He, K. (2025) 'Mean Flows for One-step Generative Modeling', *arXiv preprint* arXiv:2505.13447. Available at: https://arxiv.org/abs/2505.13447 (Accessed: 10 May 2026).

Li, T. and He, K. (2025) 'Back to Basics: Let Denoising Generative Models Denoise', *arXiv preprint* arXiv:2511.13720. Available at: https://arxiv.org/abs/2511.13720 (Accessed: 10 May 2026).

Ma, N., Goldstein, M., Albergo, M.S., Boffi, N.M., Vanden-Eijnden, E. and Xie, S. (2024) 'SiT: Exploring Flow and Diffusion-Based Generative Models with Scalable Interpolant Transformers', *Proceedings of the European Conference on Computer Vision (ECCV)*. Available at: https://arxiv.org/abs/2401.08740 (Accessed: 10 May 2026).

Peebles, W. and Xie, S. (2023) 'Scalable Diffusion Models with Transformers', *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Available at: https://arxiv.org/abs/2212.09748 (Accessed: 10 May 2026).

Zheng, B., Ma, N., Tong, S. and Xie, S. (2025) 'Diffusion Transformers with Representation Autoencoders', *arXiv preprint* arXiv:2510.11690. Available at: https://arxiv.org/abs/2510.11690 (Accessed: 10 May 2026).

Appendix

A Part 2: Full 36-figure parameterisation grid

All 36 individual generated-sample figures from the prediction × loss × dataset × dimension sweep referenced in Section 3. Order: x-pred + x-loss, x-pred + v-loss, v-pred + x-loss, v-pred + v-loss; within each block, `swiss_roll`, `gaussians`, `circles` at $D \in \{2, 8, 32\}$.

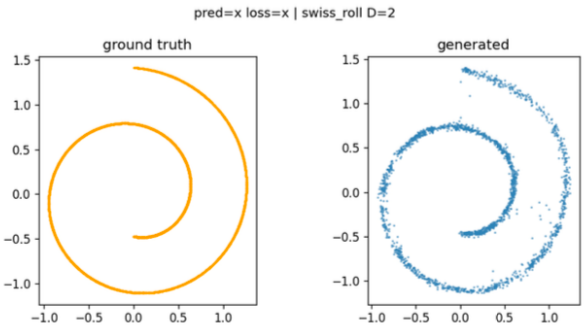


Figure 7: x-prediction with x-loss, swiss roll, D=2.

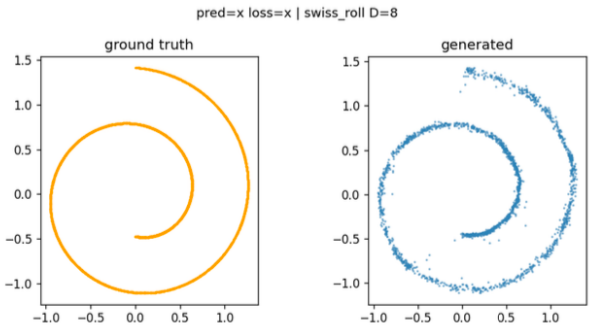


Figure 8: x-prediction with x-loss, swiss roll, D=8.

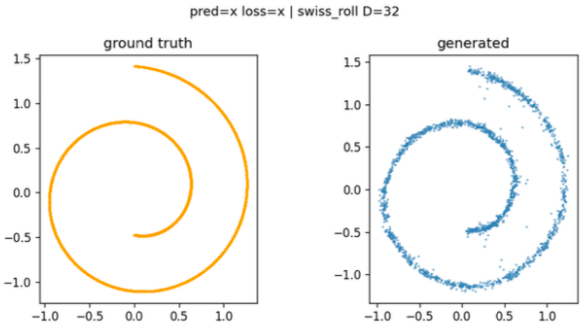


Figure 9: x-prediction with x-loss, swiss roll, D=32.

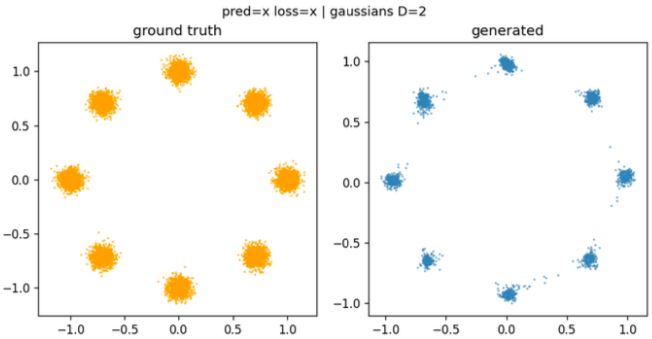


Figure 10: x-prediction with x-loss, gaussians, D=2.

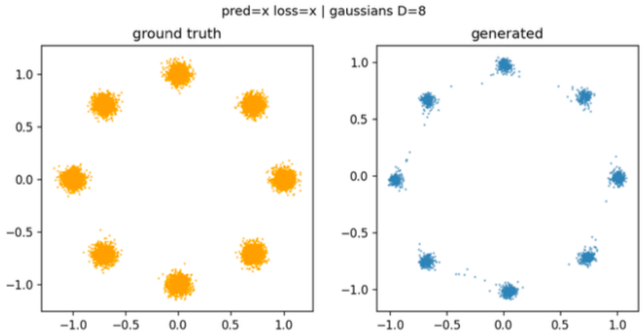


Figure 11: x-prediction with x-loss, gaussians, D=8.

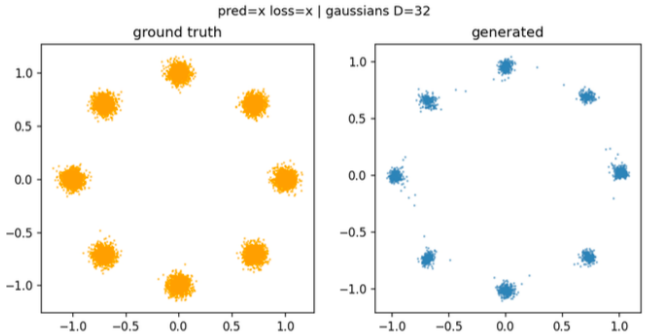


Figure 12: x-prediction with x-loss, gaussians, D=32.

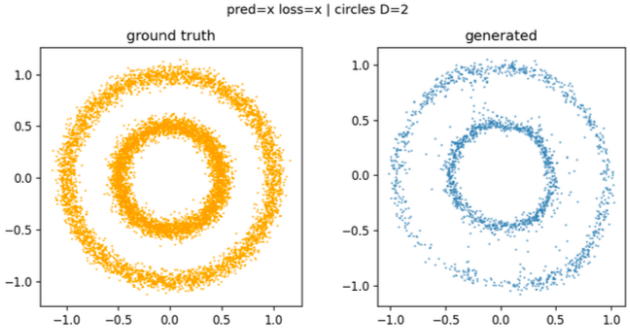


Figure 13: x-prediction with x-loss, circles, D=2.

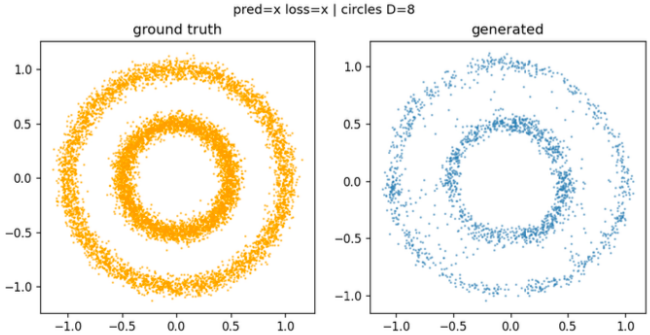


Figure 14: x-prediction with x-loss, circles, D=8.

Figure 15: x-prediction with x-loss, circles, D=32.

Figure 16: x-prediction with v-loss, swiss roll, D=2.

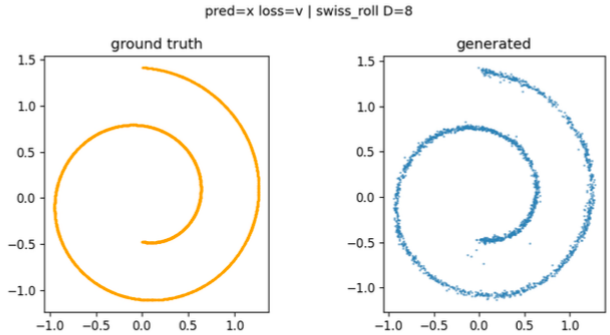


Figure 17: x-prediction with v-loss, swiss roll, D=8.

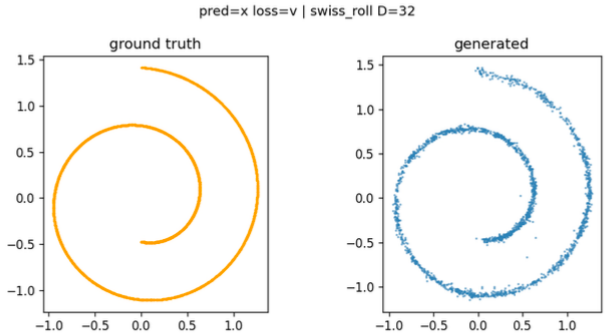


Figure 18: x-prediction with v-loss, swiss roll, D=32.

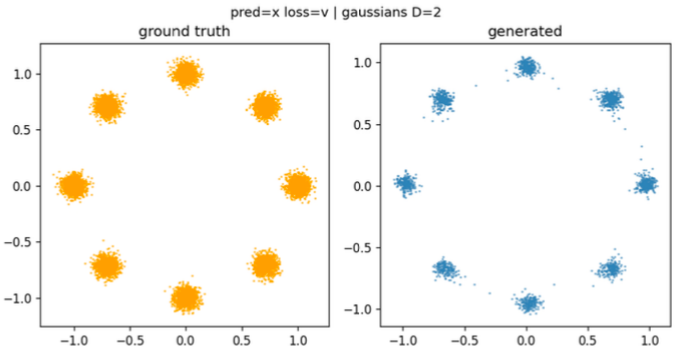


Figure 19: x-prediction with v-loss, gaussians, D=2.

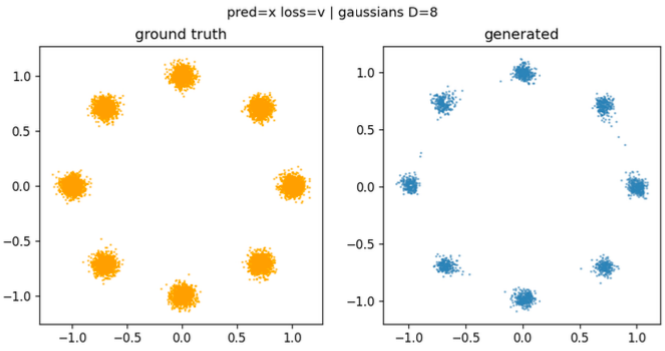


Figure 20: x-prediction with v-loss, gaussians, D=8.

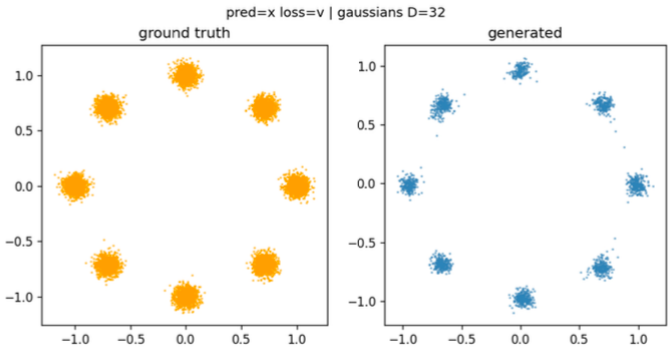


Figure 21: x-prediction with v-loss, gaussians, D=32.

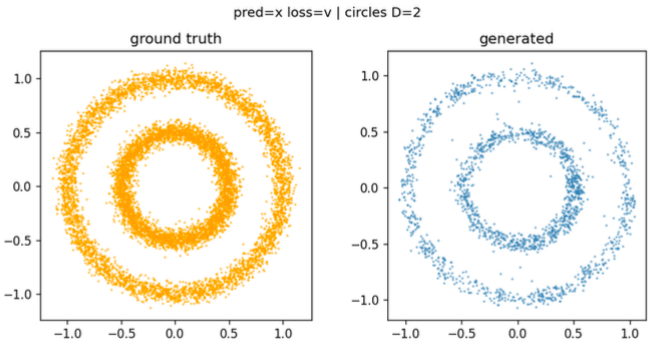


Figure 22: x-prediction with v-loss, circles, D=2.

Figure 23: x-prediction with v-loss, circles, D=8.

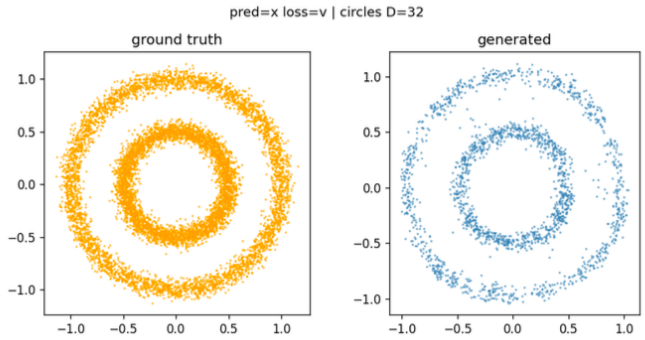


Figure 24: x-prediction with v-loss, circles, D=32.

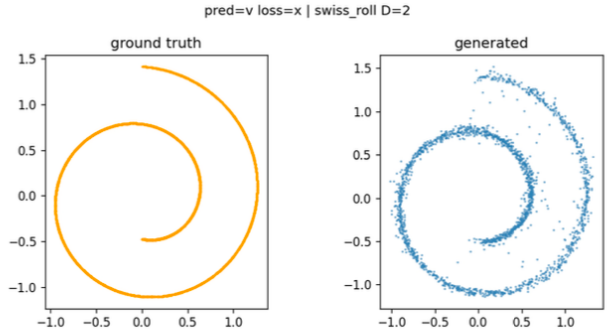


Figure 25: v-prediction with x-loss, swiss roll, D=2.

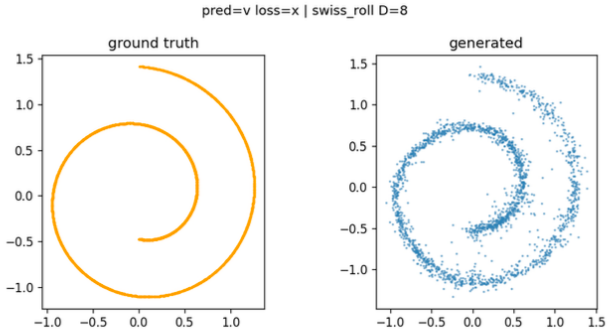


Figure 26: v-prediction with x-loss, swiss roll, D=8.

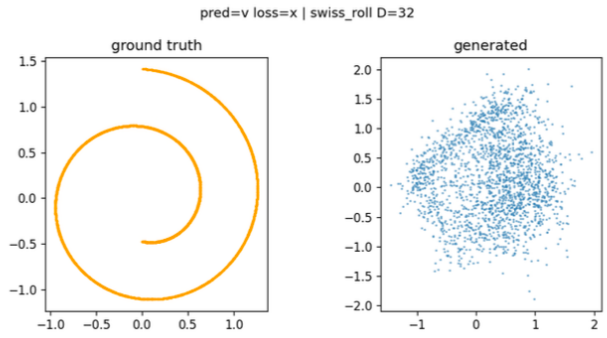


Figure 27: v-prediction with x-loss, swiss roll, D=32.

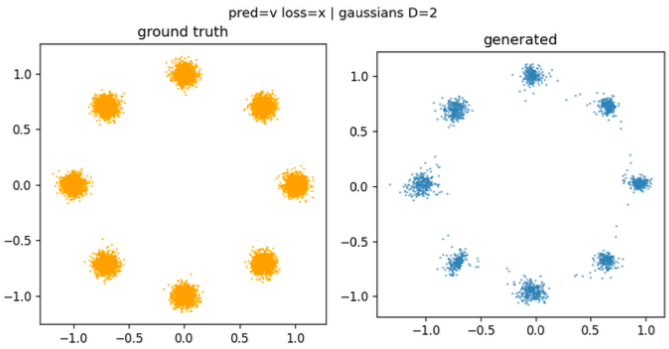


Figure 28: v-prediction with x-loss, gaussians, D=2.

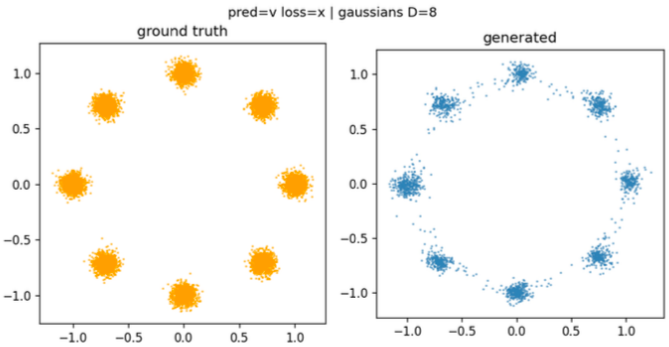


Figure 29: v-prediction with x-loss, gaussians, D=8.

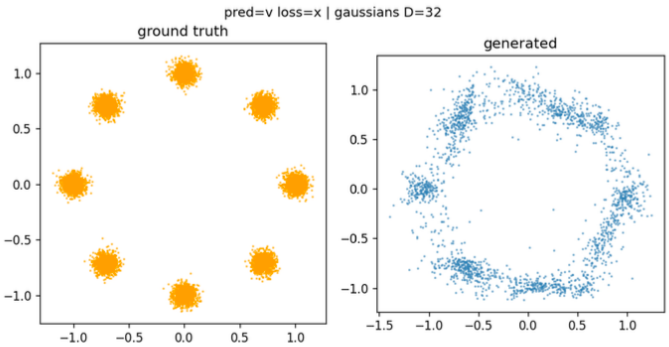


Figure 30: v-prediction with x-loss, gaussians, D=32.

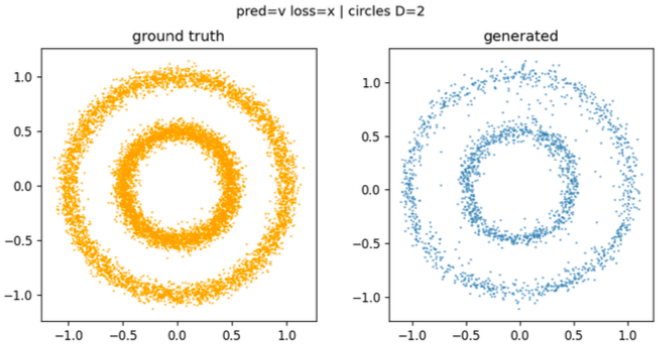


Figure 31: v-prediction with x-loss, circles, D=2.

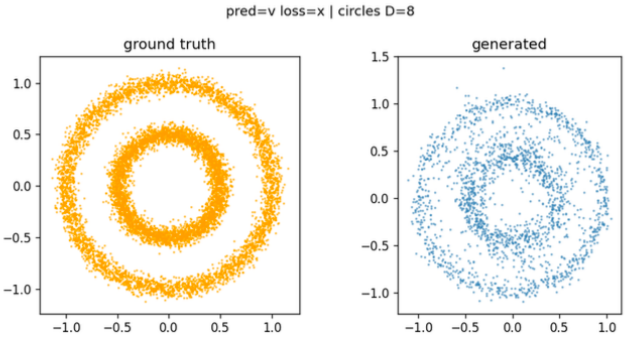


Figure 32: v-prediction with x-loss, circles, D=8.

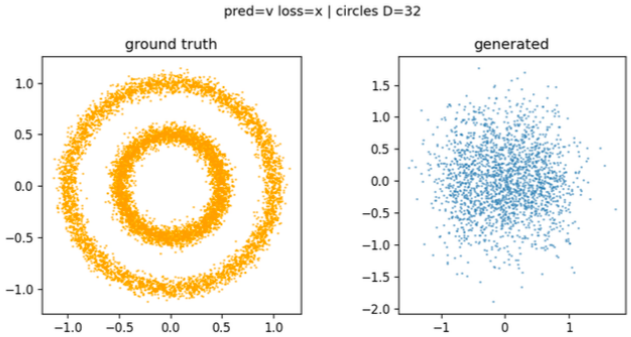


Figure 33: v-prediction with x-loss, circles, D=32.

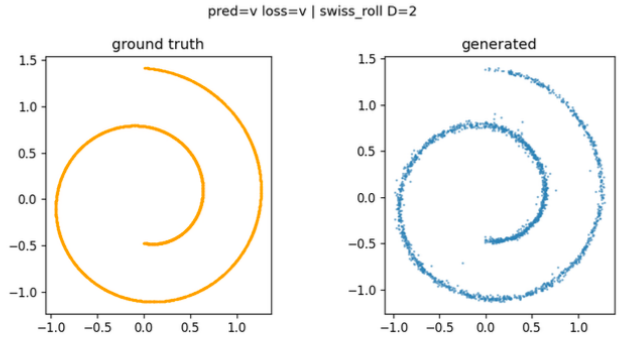


Figure 34: v-prediction with v-loss, swiss roll, D=2.

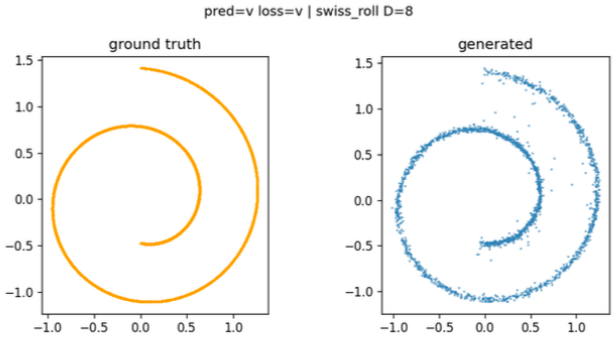


Figure 35: v-prediction with v-loss, swiss roll, D=8.

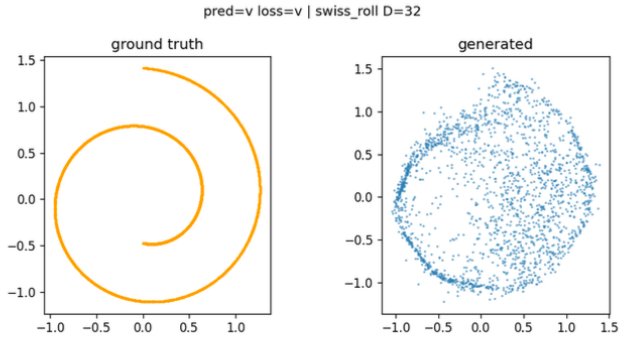


Figure 36: v-prediction with v-loss, swiss roll, D=32.

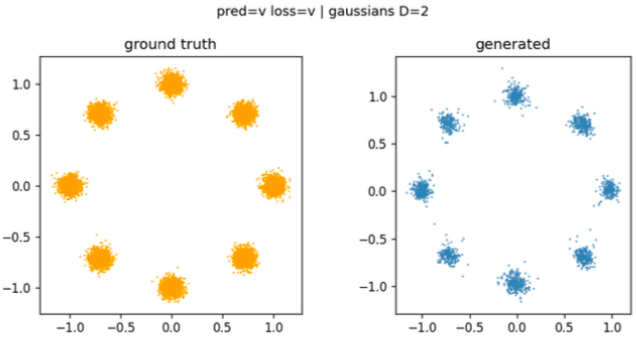


Figure 37: v-prediction with v-loss, gaussians, D=2.

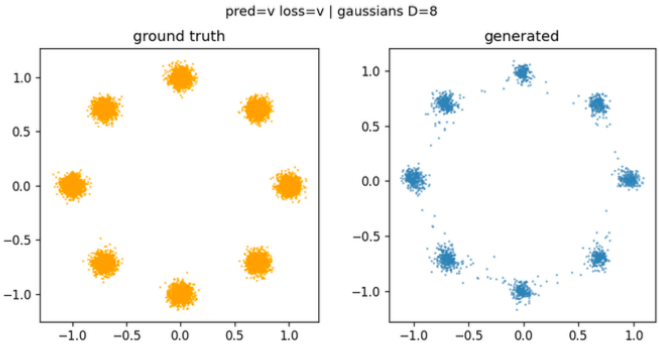


Figure 38: v-prediction with v-loss, gaussians, D=8.

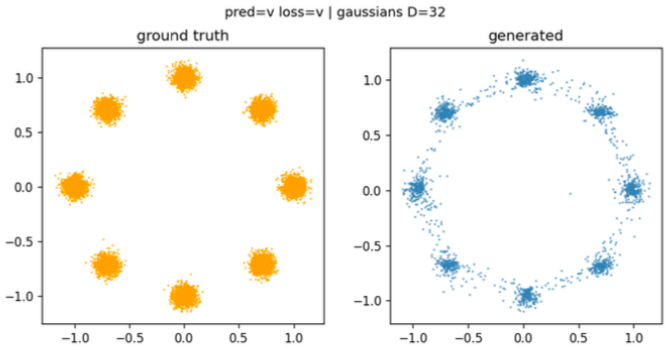


Figure 39: v-prediction with v-loss, gaussians, D=32.

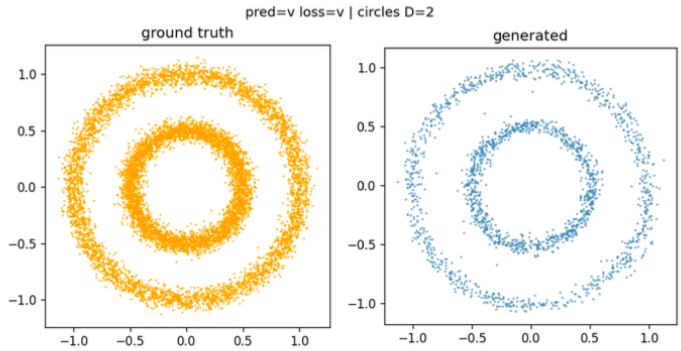


Figure 40: v-prediction with v-loss, circles, D=2.

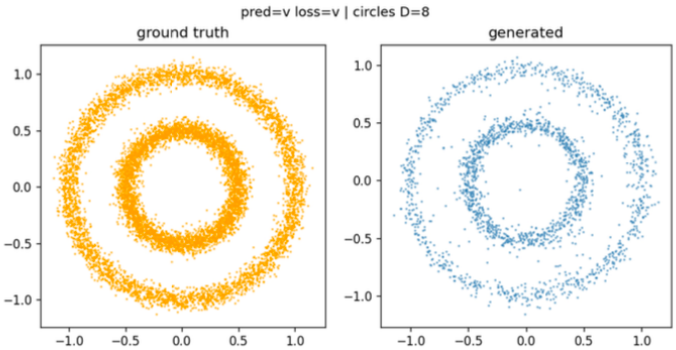


Figure 41: v-prediction with v-loss, circles, D=8.

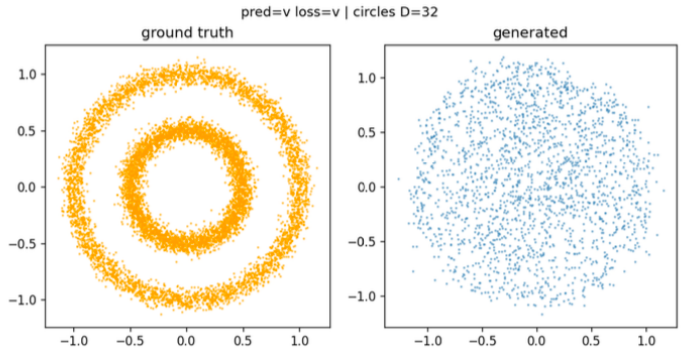


Figure 42: v-prediction with v-loss, circles, D=32.

B Part 3: Width × schedule sweep on `swiss_roll`

All seven configurations from the v-prediction rescue sweep at $D = 32$, referenced in Section 4. Width $h \in \{256, 512, 1024\}$ under default and shifted schedules, plus the x-prediction reference at $h = 256$.

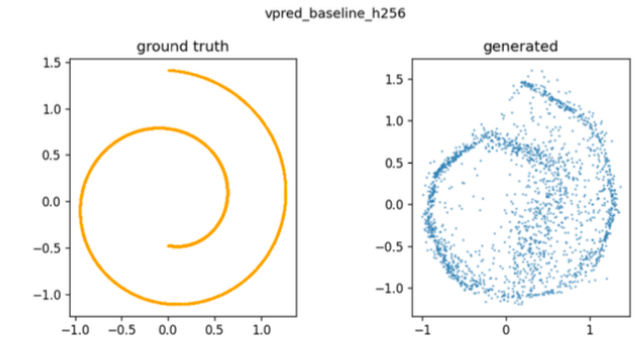


Figure 43: v-prediction, h=256, default schedule (failing baseline). `swiss_roll`, D=32.

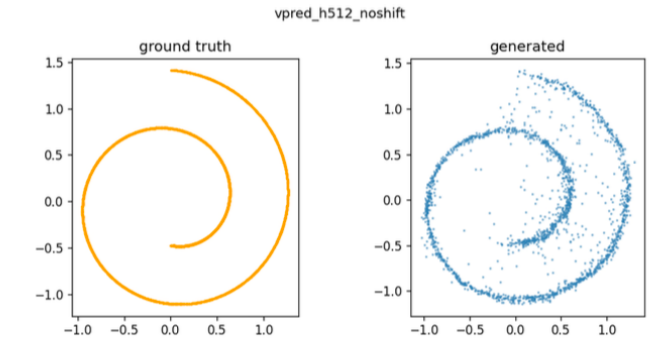


Figure 44: v-prediction, h=512, default schedule. `swiss_roll`, D=32.

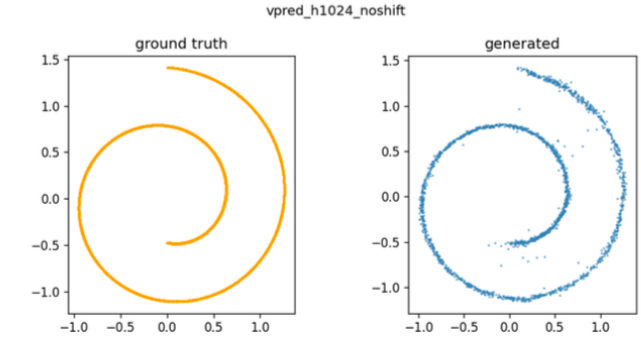


Figure 45: v-prediction, h=1024, default schedule (rescued). `swiss_roll`, D=32.

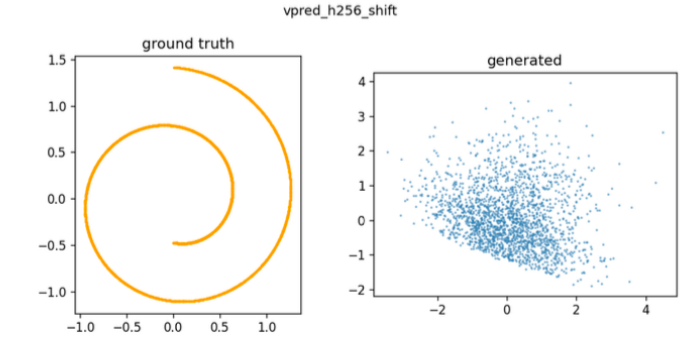


Figure 46: v-prediction, h=256, shifted schedule. `swiss_roll`, D=32.

Figure 47: v-prediction, h=512, shifted schedule. `swiss_roll`, D=32.

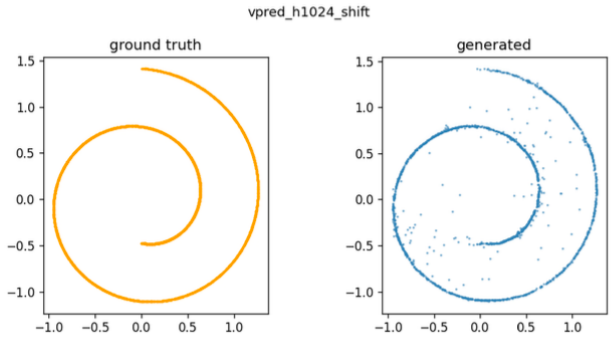


Figure 48: v-prediction, h=1024, shifted schedule. `swiss_roll`, D=32.

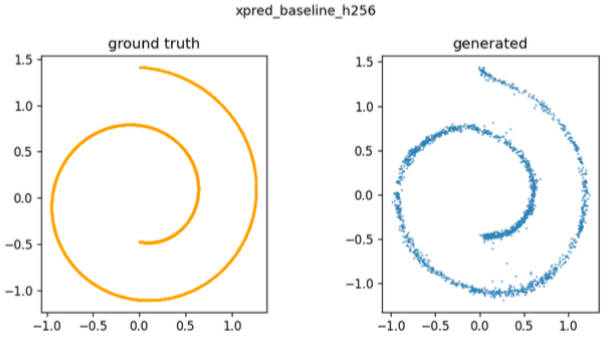


Figure 49: x-prediction, h=256, default schedule (reference). `swiss_roll`, D=32.

C Part 4.1: Full step-count sweep

All 27 figures from the Euler step-count sweep at $D = 32$ referenced in Section 5. Per-dataset summary grids first, then individual figures at every Euler step count $\{1, 2, 5, 10, 20, 50, 100, 200\}$ for each dataset.

C.1 Per-dataset summary grids

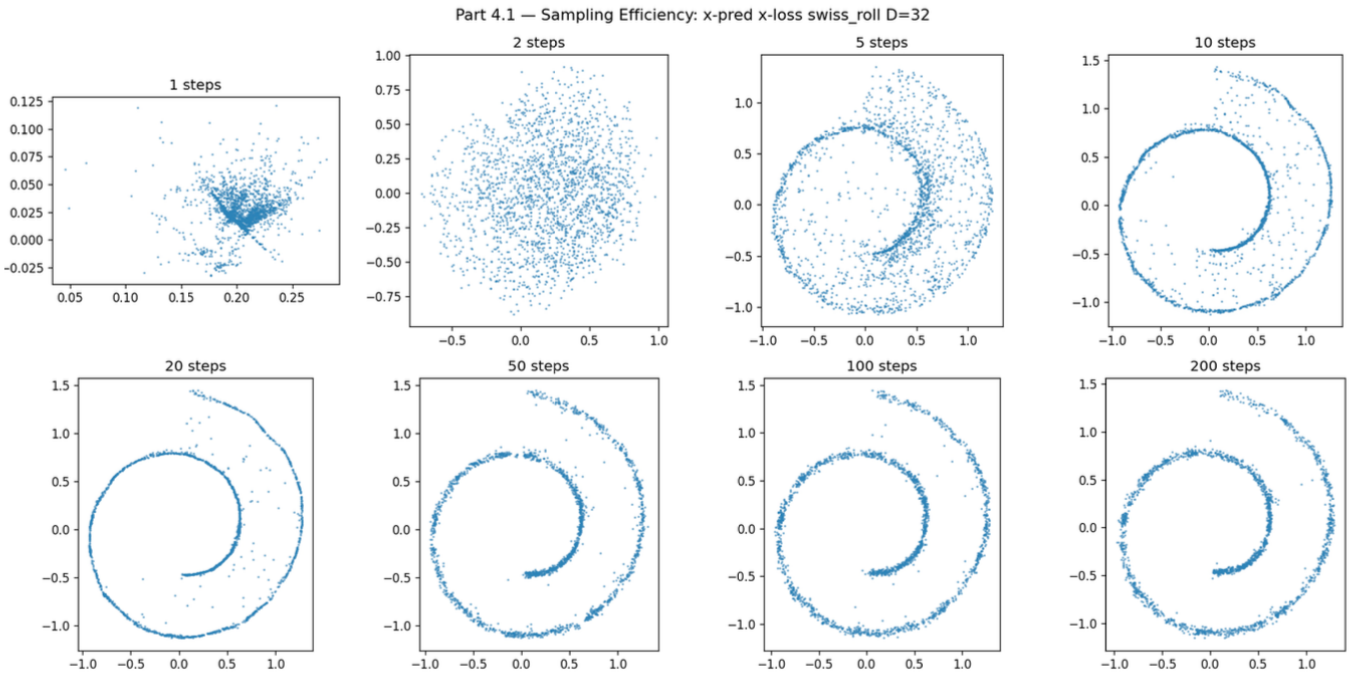


Figure 50: Step-count summary, swiss roll, D=32. Generated samples at Euler step counts $\{1, 2, 5, 10, 20, 50, 100, 200\}$.

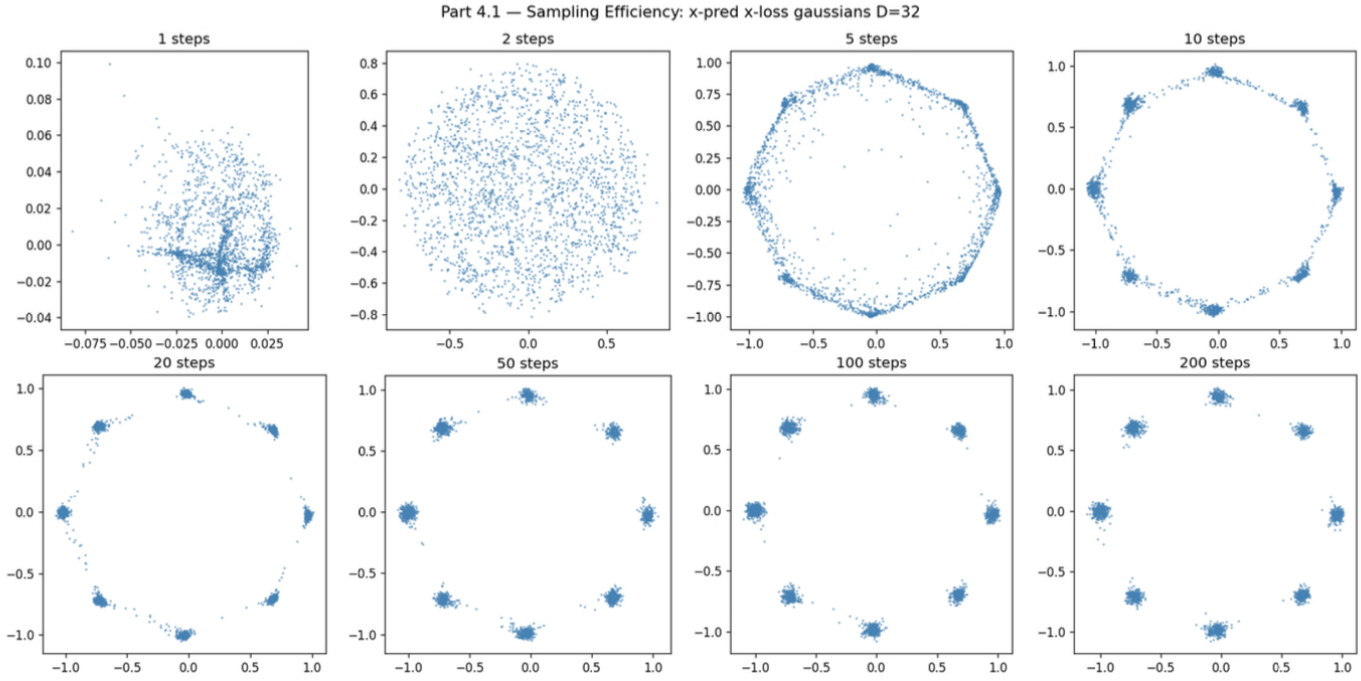


Figure 51: Step-count summary, gaussians, D=32. Generated samples at Euler step counts $\{1, 2, 5, 10, 20, 50, 100, 200\}$.

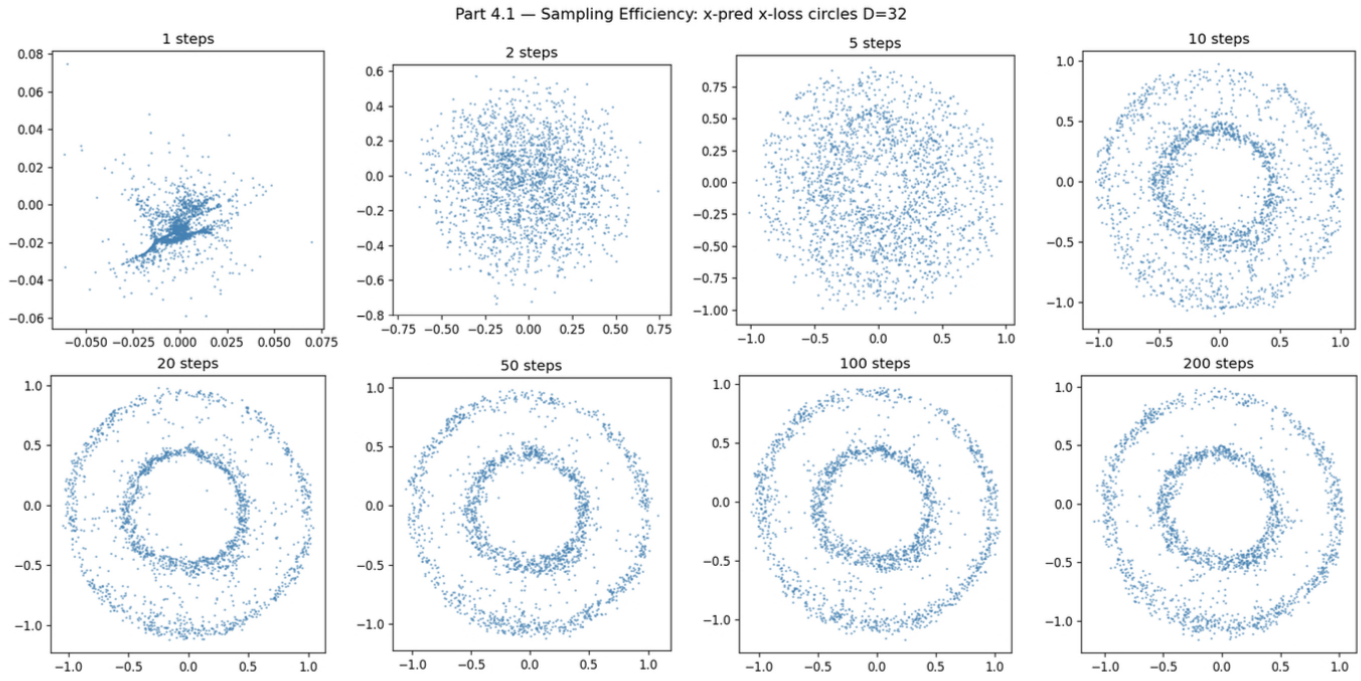


Figure 52: Step-count summary, circles, D=32. Generated samples at Euler step counts $\{1, 2, 5, 10, 20, 50, 100, 200\}$.

C.2 Individual step-count figures

`swiss_roll`, D=32

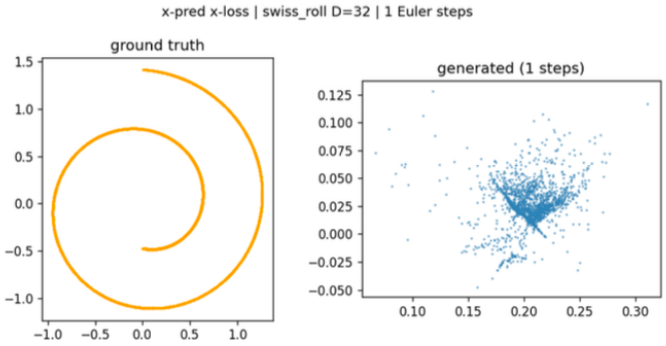


Figure 53: swiss roll, D=32, 1 Euler step.

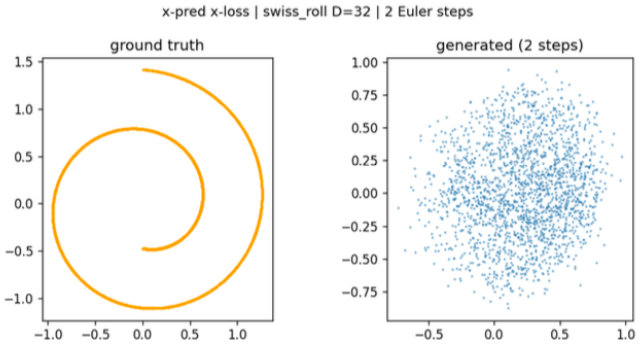


Figure 54: swiss roll, D=32, 2 Euler steps.

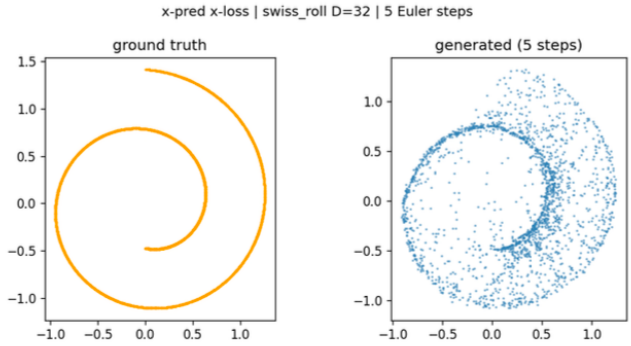


Figure 55: swiss roll, D=32, 5 Euler steps.

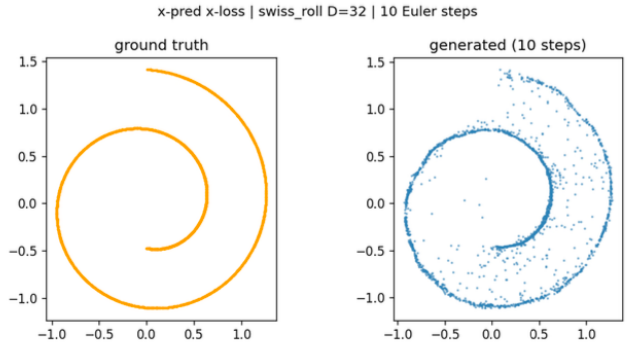


Figure 56: swiss roll, D=32, 10 Euler steps.

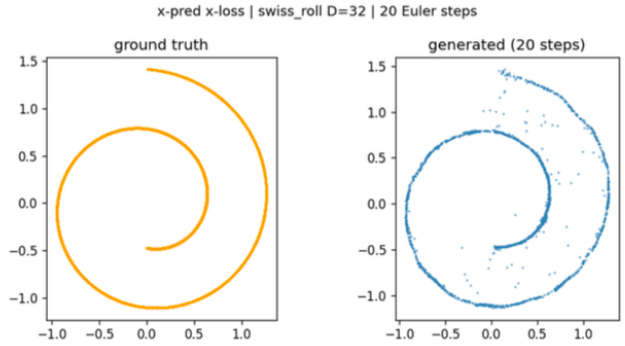


Figure 57: swiss roll, D=32, 20 Euler steps.

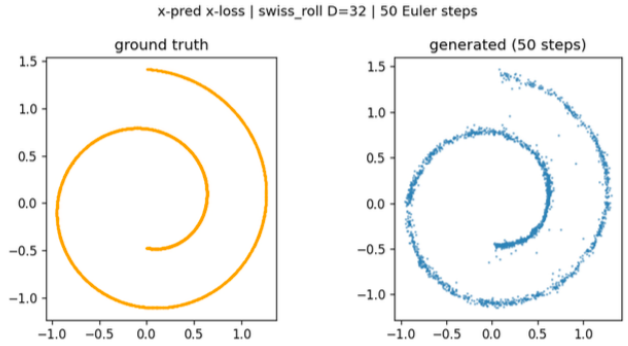


Figure 58: swiss roll, D=32, 50 Euler steps.

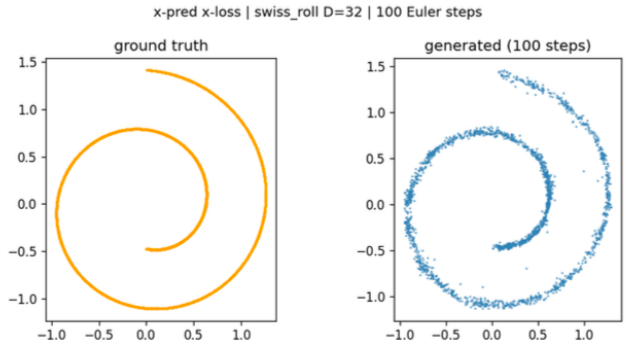


Figure 59: swiss roll, D=32, 100 Euler steps.

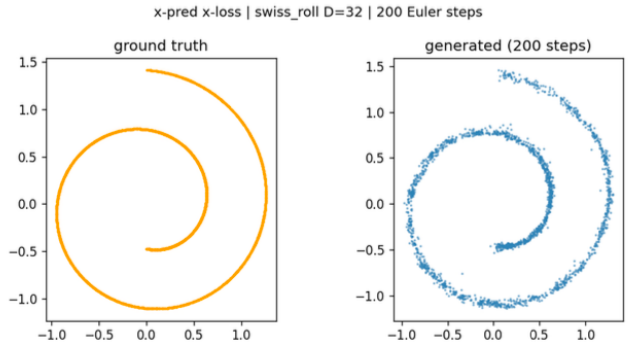


Figure 60: swiss roll, D=32, 200 Euler steps.

gaussians, D=32

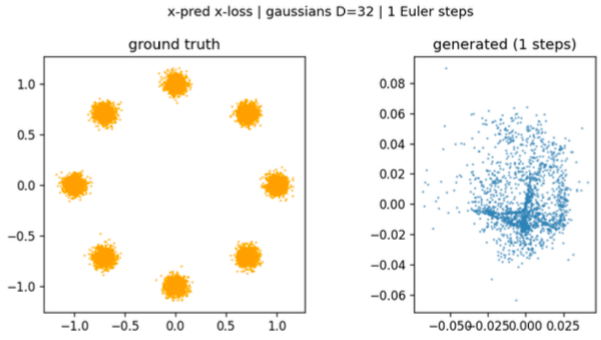


Figure 61: gaussians, D=32, 1 Euler step.

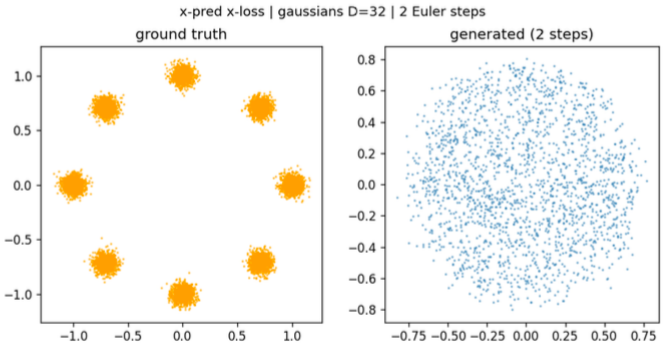


Figure 62: gaussians, D=32, 2 Euler steps.

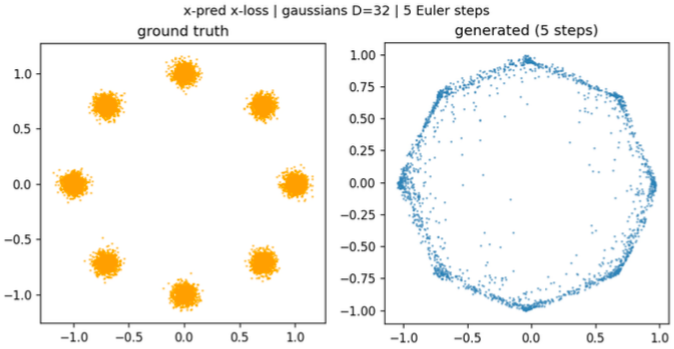


Figure 63: gaussians, D=32, 5 Euler steps.

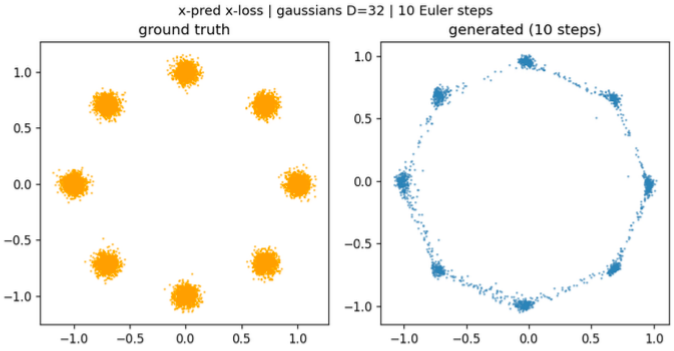


Figure 64: gaussians, D=32, 10 Euler steps.

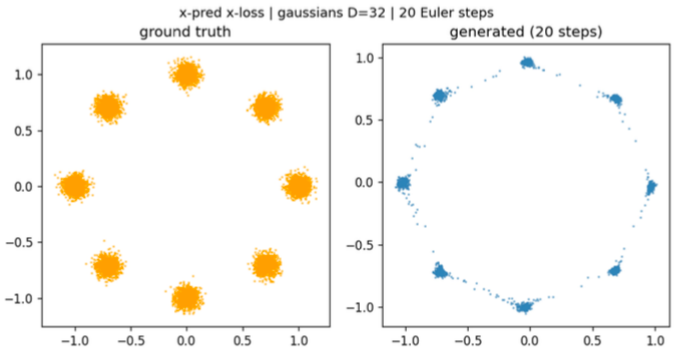


Figure 65: gaussians, D=32, 20 Euler steps.

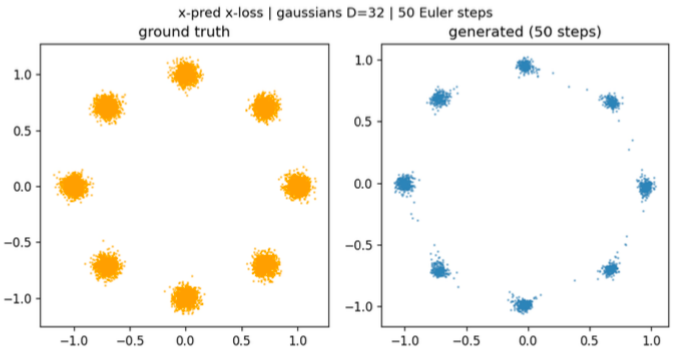


Figure 66: gaussians, D=32, 50 Euler steps.

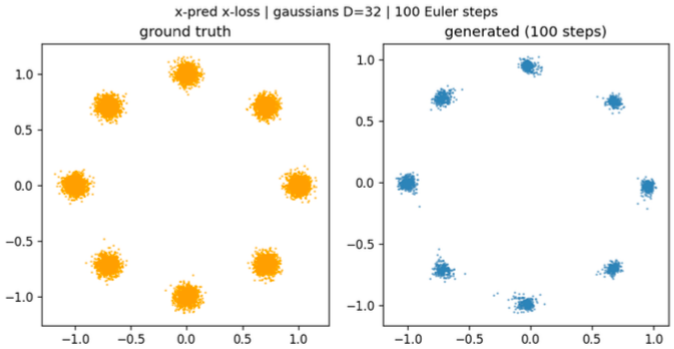


Figure 67: gaussians, D=32, 100 Euler steps.

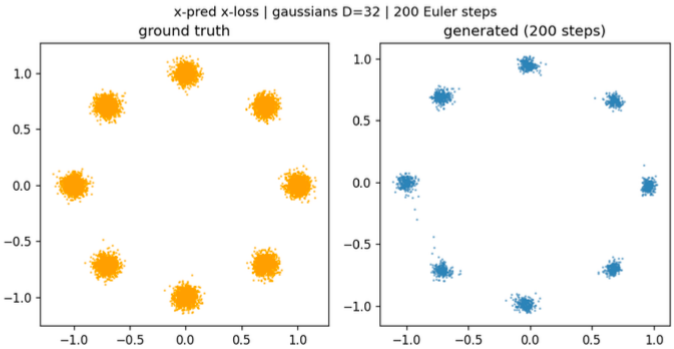


Figure 68: gaussians, D=32, 200 Euler steps.

`circles`, D=32

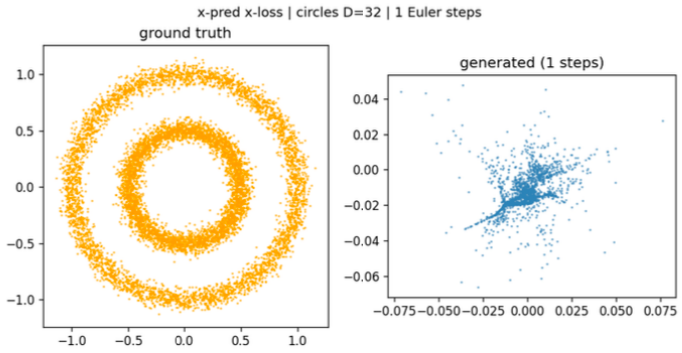


Figure 69: circles, D=32, 1 Euler step.

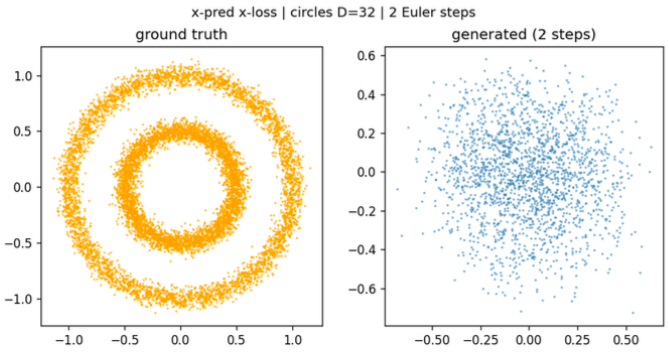


Figure 70: circles, D=32, 2 Euler steps.

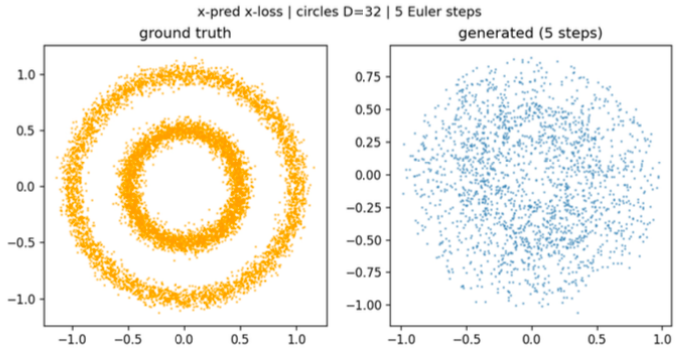


Figure 71: circles, D=32, 5 Euler steps.

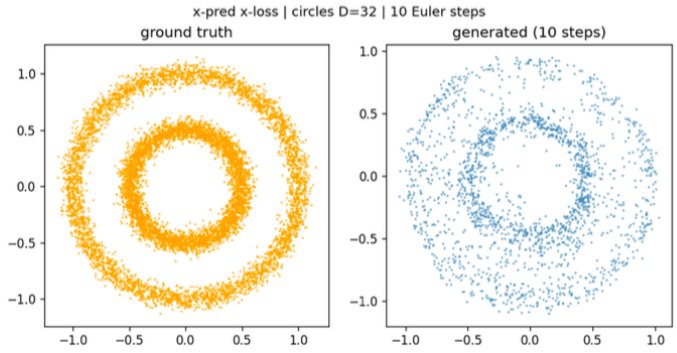


Figure 72: circles, D=32, 10 Euler steps.

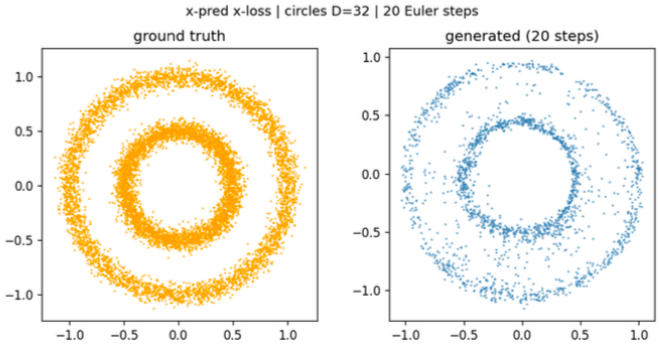


Figure 73: circles, D=32, 20 Euler steps.

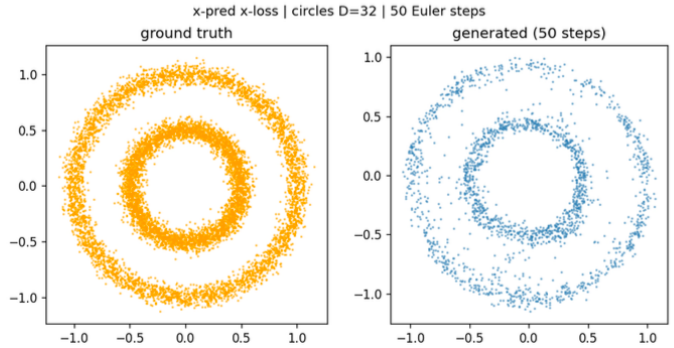


Figure 74: circles, D=32, 50 Euler steps.

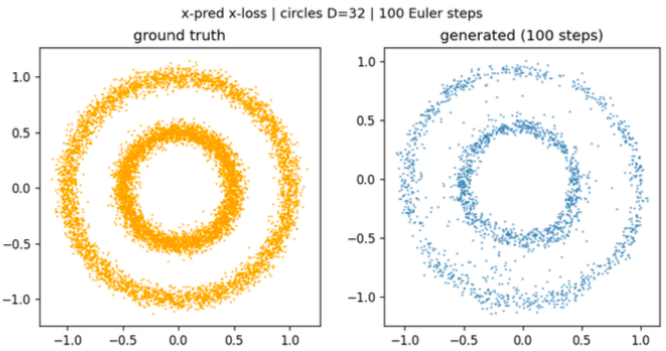


Figure 75: circles, D=32, 100 Euler steps.

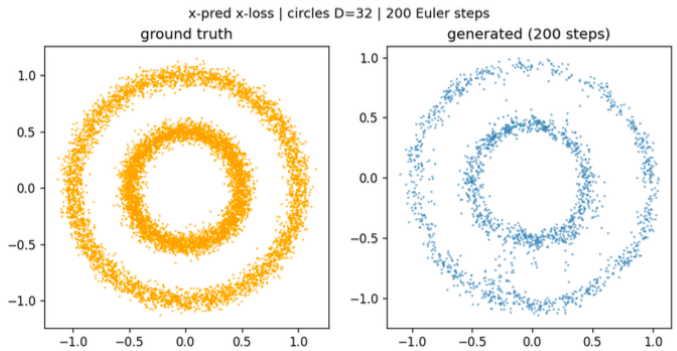


Figure 76: circles, D=32, 200 Euler steps.

D Part 4.2: All MeanFlow generated samples

All nine MeanFlow figures referenced in Section 6, organised by dataset. Each row shows MeanFlow generated samples at $\{1, 2, 5\}$ steps for one dataset at $D = 32$. Per-dataset seeds are those selected from the 10-seed sensitivity sweep (0 for `swiss_roll`, 100 for `gaussians`, 456 for `circles`).

`swiss_roll`, D=32

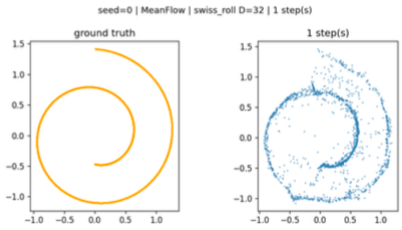


Figure 77: swiss roll, 1 MeanFlow step.

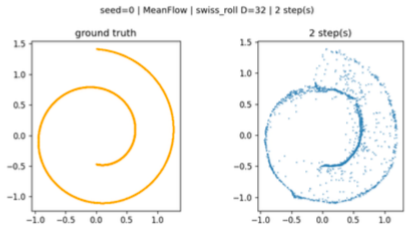


Figure 78: swiss roll, 2 MeanFlow steps.

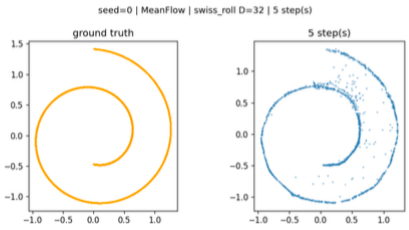


Figure 79: swiss roll, 5 MeanFlow steps.

`gaussians`, D=32

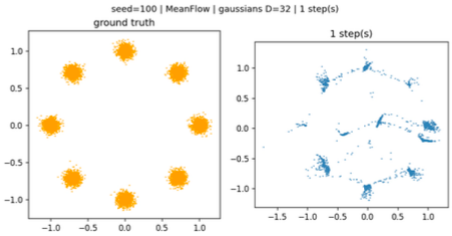


Figure 80: gaussians, 1 MeanFlow step.

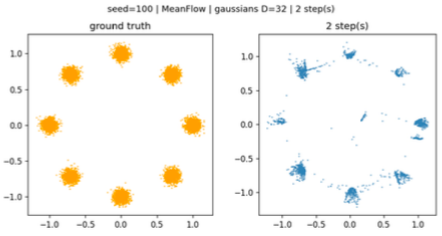


Figure 81: gaussians, 2 MeanFlow steps.

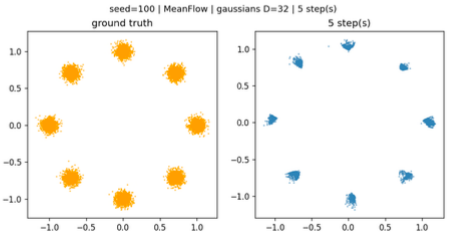


Figure 82: gaussians, 5 MeanFlow steps.

`circles`, D=32

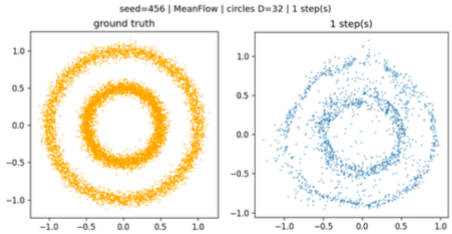


Figure 83: circles, 1 MeanFlow step.

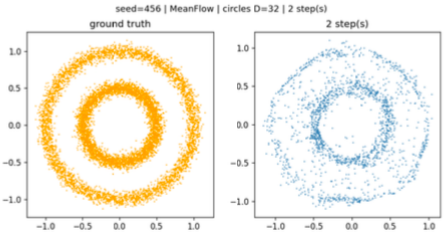


Figure 84: circles, 2 MeanFlow steps.

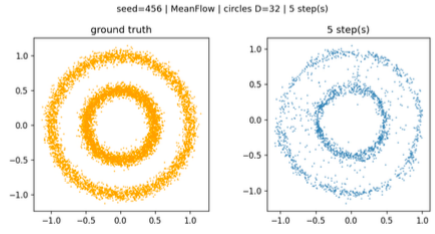


Figure 85: circles, 5 MeanFlow steps.